Component systems: variants, states, and reuse

Week 7, Lecture 2 · Design for Builders: Ship Beautiful Products as a Founder

Autumn 2026 · Founders, indie hackers, engineers

What we'll cover today

- **Atomic design as a mental model:** why atoms, molecules, and organisms map to Figma
- **Buttons with states:** building the component every screen shares
- **Form inputs with variants:** text, select, checkbox, and their interaction states
- **Cards as a primitive:** one component that becomes many things
- **Modals and overlays:** when to use them and how to design them safely

Atomic design as a mental model

From atoms to pages

Brad Frost (2016) describes five levels in any design system:

```
Atoms       Tokens: color, type, spacing, icons
Molecules   Single-purpose components: button, input, badge
Organisms   Section-level composites: form, card list, nav bar
Templates   Screen layouts without real content
Pages       Templates filled with real content
```

The insight is not the vocabulary. It is that design systems are not built top-down. You define atoms first, combine them into molecules, and only then assemble organisms. A button is a molecule; a checkout form is an organism.

Why this matters for founders

Most founders start with screens (pages) and work backward to components only when something looks inconsistent.

Working backward costs time every time you edit: change a button color in one place, miss it in three others.

Working forward costs time once: define the button, use it everywhere, change it once.

Figma's component system enforces the forward approach. A master component and its instances mean one change propagates everywhere.

Buttons

The button is the contract with your user

Every interactive decision in your product passes through a button. Its states are the visual contract: "this is clickable, this is not, this is loading, this is done."

Required states:

- **Default:** the neutral, ready state
- **Hover:** confirmation that the element is interactive
- **Pressed:** acknowledgment that the click registered
- **Disabled:** the action is unavailable, and the reason should be nearby
- **Loading:** the action was triggered, the result is pending

(Figma Learn, design an interactive button component, 2023)

Building the button in Figma

```
Component structure:
  Button (auto-layout, horizontal)
    ├── [Optional] Leading icon
    ├── Label (text layer, token: type/label/md)
    └── [Optional] Trailing icon

Variant properties:
  Hierarchy: Primary | Secondary | Ghost
  Size:      sm | md | lg
  State:     Default | Hover | Pressed | Disabled | Loading
```

Every fill and stroke references a semantic color token. "Button/surface/primary/default" beats "#0050FF" because a token survives a palette change; a hardcoded hex does not.

The disabled state trap

Disabled buttons are often designed as just a faded version of the primary button. That is insufficient.

A faded button tells the user: "this button exists but you cannot use it." It does not tell them why.

- Use a color that passes a 3:1 contrast ratio for the button surface against the page background (WCAG 1.4.11, non-text contrast)
- Place the reason for disabling near the button: a tooltip on hover, a message above the form, or an inline label
- Consider whether disabling is even the right pattern, or whether validation errors on submit would serve the user better

Form inputs

Every input has four states at minimum

```
State      Visual signal
Default    Neutral border, no fill or subtle fill, label above
Focused    Accent color border (2px), label stays above
Error      Error color border, error message below in error color
Disabled   Muted fill, muted label, no interactive cursor
```

A fifth state, Success (green border, checkmark), is useful for email or username fields where real-time validation fires on blur. Do not use success states on every field: it creates noise.

Label placement

There are three common label patterns:

1. **Label above input** (recommended): always visible, works at all viewport widths
2. **Floating label** (animates from placeholder to label on focus): works visually but creates a two-state confusion problem if the user cannot tell whether the field is empty or pre-filled
3. **Placeholder only** (no label): fails for users who need to review their entries before submitting, and disappears once they type

Default to labels above inputs. The floating label has a place in single-field, high-design contexts only.

Checkbox and radio: the forgotten inputs

Checkboxes and radios break in most component libraries because designers copy them from a UI kit without building proper variants.

Required states for both:

- Unchecked/default
- Checked
- Indeterminate (checkbox only, for parent-child selections)
- Error (red border if required and skipped)
- Disabled unchecked
- Disabled checked

The check mark and the radio dot must be part of the component, not drawn on-screen per instance.

Cards

The card as a primitive

A card is a bounded content container. It is the most overloaded component in product design because it can become:

- A list item (user, document, task)
- A dashboard tile (metric, chart, status)
- A product grid item (e-commerce, gallery)
- A notification item
- A modal header block

The reason one component can do all of these is that a card is structure, not content. The structure (rounded corners, shadow, padding, content slot) is what you build. The content varies by context.

Building a flexible card

```
Component structure:
  Card (auto-layout, vertical, hug height)
    ├── [Optional] Image slot (fixed height, hidden by boolean)
    ├── Content area (auto-layout, vertical, padding: space-4)
    │     ├── Title (type/heading/sm token)
    │     ├── Body (type/body/md token)
    │     └── [Optional] Meta row (icon + label)
    └── [Optional] Action row (hidden by boolean)
          └── Ghost button

Boolean properties:  Has image | Has action | Has meta
```

With two boolean properties, one master component produces four usable configurations without any duplicated frames.

(Frost, Atomic Design ch. 2, 2016; Figma, components best practices, 2023)

Cards and color tokens

The card surface color is one of the most common semantic token uses.

- `color/surface/card` for the card background (slightly off-white in light mode, slightly elevated in dark mode)
- `color/border/subtle` for the card border if you use one
- `color/shadow/sm` for the card drop shadow

Hardcoding `#FFFFFF` for the card surface means your dark mode is broken from the start. Figma (Miao, 2022) built their entire dark mode on semantic tokens for exactly this reason.

Modals and overlays

What a modal is for

A modal is an interruption. Use it when:

- The user must make a decision before continuing (destructive confirmation)
- A focused task benefits from full attention without navigation away (quick add, edit form)
- The current page state should be preserved underneath

Do not use a modal when:

- The task is long or complex (use a new page or a drawer)
- The content is primarily informational (use an inline expansion or a tooltip)
- The user initiated a navigation action (use the destination page)

Modal structure

```
Component structure:
  Backdrop (full-viewport overlay, 50% opacity black)
  Modal (auto-layout, vertical, centered, max-width: 480px)
    ├── Header (auto-layout, horizontal)
    │     ├── Title (type/heading/md)
    │     └── Close button (icon button variant)
    ├── Body (auto-layout, vertical, padding: space-6)
    │     └── Content slot
    └── Action row (auto-layout, horizontal, reversed)
          ├── Secondary button
          └── Primary button
```

Action buttons are right-aligned on desktop. The primary action is rightmost (closer to where the user's eye lands after reading).

The focus trap problem

A modal without a focus trap is an accessibility failure. When the modal opens, keyboard focus must move inside the modal and stay inside until the modal closes.

In Figma, you cannot prototype a focus trap. But you can design for it:

- The first focusable element inside the modal should be the title or the first input, not the close button
- The close button (X) should be the last focusable element in the tab order
- Pressing Escape should close the modal (design a keyboard shortcut note in your spec annotations)

This is a handoff concern, not a visual design concern. Document it in your component spec.

Take-aways

1. Atomic design means building upward: tokens first, molecules second, organisms third. Working backward from screens produces inconsistency.
2. A button with five states is a complete component. A button with one state is a rectangle with rounded corners.
3. Cards are structure, not content. One flexible card master component with boolean properties replaces four duplicated frames.
4. Modals interrupt. Use them only when interruption serves the user. Document focus-trap behavior in your handoff spec even though Figma cannot prototype it.

> "What makes a design system successful is not the components themselves but the decisions that are baked into them."

Figma, Components, Styles, and Shared Library Best Practices (2023)

What's next

- **Section (this week):** build a five-screen flow using your component library
- **Reading:** Week 7 reading, app screens, states, and component systems
- **Week 8:** brand identity, logo, wordmark, and the system around them

HW 3 due this week: full landing page design.
Capstone out this week: launch package.